Automatic Speech Recognition

Laboratory Work 3: Fine-tuning Whisper on the Toronto Ukrainian Dataset

Matvii Shumylovych, Andrii Kravchuck, Student Edvard

April 24, 2026

1 Introduction

The goal of this lab is to build an Automatic Speech Recognition (ASR) system for the Ukrainian language using the Toronto dataset. We had to pick a pre-trained Whisper model from Hugging Face, write a training loop with PyTorch Lightning, fine-tune the model, and then test it on a fixed held-out test set using WER and CER metrics.

The Toronto dataset contains Ukrainian audio recordings paired with text transcriptions. The data is organized into groups called "lines", where each line is a set of short utterances from the same source/speaker session.

2 Exploratory Data Analysis

Before doing anything with the model, we first looked at the dataset to understand what we are working with. The labels file contained 29,232 audio–text pairs in total.

2.1 Dataset Statistics

Table 1: Toronto dataset overall statistics

Statistic	**Value**
Total audio–text pairs (labels)	29,232
Unique meaningful words	63,431
Average words per phrase	14.04
Maximum words in a phrase	65
Minimum words in a phrase	0
Phrases containing digits	2,538

The average phrase is about 14 words long, which is a pretty typical sentence length. We also noticed that 2,538 phrases have digits in them (like "2024" or "5 thousand"). In spoken Ukrainian, numbers are said as words, so this could cause some issues when comparing predictions to references. We did not apply number normalization in this work, only basic whitespace cleanup.

2.2 Transcript Length Distribution

Figure 1 shows how long the transcripts are (in words). Most phrases are between 10 and 20 words, with a peak around 13–14.

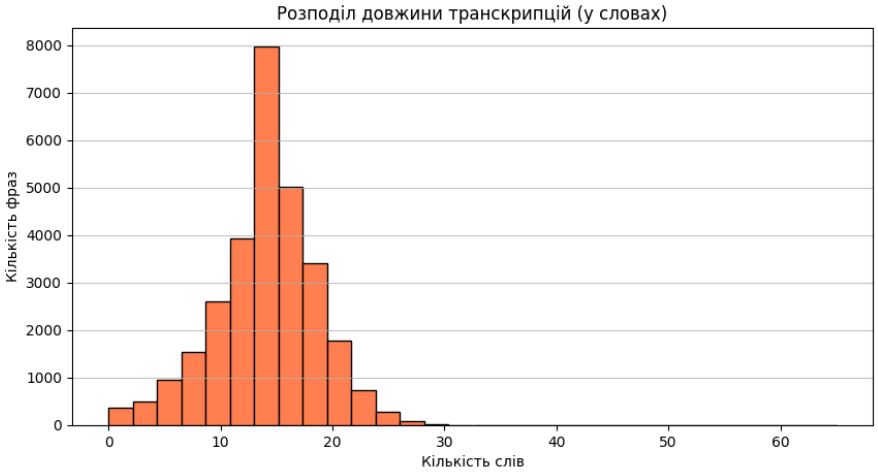

Figure 1: Distribution of transcript lengths (in words) across the Toronto dataset.

2.3 Audio Files

Even though the labels file has 29,232 entries, only 18,303 WAV files were actually present in the dataset. After matching audio files to their labels, we got 18,199 valid samples. The other 10,929 entries were just missing their audio, so we skipped them. All audio passed the 30-second maximum duration filter.

Table 2: Audio duration statistics (18,199 samples)

Statistic	**Value**
Mean duration	5.96 s
Std deviation	1.24 s
Minimum	0.14 s
25th percentile	5.31 s
Median	6.20 s
75th percentile	6.88 s
Maximum	16.02 s

Clips are pretty short - mostly around 6 seconds - which is normal for ASR datasets where each utterance is one sentence.

3 Data Splitting

To make sure there is no data leakage between splits, we split the data at the *line level*, not the utterance level. This means all clips from the same Toronto line always go into the same split. The 21 test lines given in the assignment were held out and never touched during training.

The remaining lines were shuffled (seed 42) and 10% went to validation, the rest to training.

Table 3: Dataset split summary

Split	Samples	Duration (h)	Unique lines
Train	11,424	18.91	45
Validation	1,258	2.07	5
Test	5,517	9.13	21
Total	**18,199**	**30.11**	**71**

We also added assertions in the code to double-check that none of the line IDs appear in more than one split.

4 Model Selection

We chose **openai/whisper-small** as the base model. Here is why:

- It has 241 million parameters, which is a good balance between performance and the ability to actually train it on a single T4 GPU without running out of memory.
- Whisper was pre-trained on a large multilingual dataset that includes Ukrainian, so it already knows the language to some degree before we even start fine-tuning.
- Bigger variants like whisper-medium or whisper-large would take way too long and probably not fit in memory without more aggressive optimizations.

The model has a standard encoder-decoder Transformer architecture:

- **Encoder**: 12 Transformer layers, hidden size 768, with two convolutional layers that take 80-channel log-Mel spectrograms as input.
- **Decoder**: 12 Transformer layers, vocabulary size 51,865.
- **Total**: 241M parameters, essentially all trainable (we did not freeze the encoder).

We loaded the model with `language="ukrainian"` and `task="transcribe"` so it always transcribes in Ukrainian.

5 Implementation

5.1 Data Pipeline

Each audio file was loaded with `torchaudio` and resampled to 16,000 Hz if needed. The Whisper feature extractor then converts the waveform into an 80-band log-Mel spectrogram of 3,000 frames (equivalent to 30 seconds, with padding or truncation). The text is tokenized with the Whisper tokenizer.

We wrote a custom `WhisperCollator` to handle batching: it pads both the spectrogram features and the label token sequences, and sets pad positions in labels to -100 so the cross-entropy loss ignores them.

For text normalization we only did the minimum:

- Unified all apostrophe variants to a standard apostrophe.
- Collapsed repeated spaces and stripped edges.

5.2 PyTorch Lightning Module

All the training logic lives in a `WhisperLightningModule` class. The main parts:

- **Training step**: runs a forward pass through Whisper and returns the cross-entropy loss.
- **Validation/test step**: same as training but also runs greedy decoding (`num_beams=1`) and computes WER and CER with the `evaluate` library.
- **Optimizer**: AdamW with lr=10^{-5} and weight decay=10^{-2}.
- **Scheduler**: linear warmup for the first 5% of steps, then linear decay to zero.

5.3 Training Configuration

Table 4: Training hyperparameters

Hyperparameter	**Value**
Base model	`openai/whisper-small`
Language / task	Ukrainian / transcribe
Hardware	Tesla T4 GPU
Precision	16-bit mixed (AMP)
Per-device batch size	4
Gradient accumulation steps	4
Effective batch size	16
Learning rate	1×10^{-5}
Weight decay	1×10^{-2}
Warmup ratio	5%
Gradient clipping	1.0
Max epochs	5
Early stopping patience	2 (on `val_wer`)
Encoder frozen	No
Random seed	42

We used three callbacks: **ModelCheckpoint** (saves best by `val_wer`), **EarlyStopping** (stops if no improvement for 2 epochs), and **LearningRateMonitor**. Metrics were logged to CSV with `CSVLogger`.

6 Results

6.1 Training Curves

Training ran for all 5 epochs without triggering early stopping. Figure 2 shows how training and validation loss changed over epochs. The training loss drops consistently,

but validation loss starts going up after epoch 1 - a clear sign of overfitting. Still, the validation WER and CER (Figure 3) kept improving through epoch 2 and then stayed roughly stable.

Figure 2: Training vs validation loss over 5 epochs.

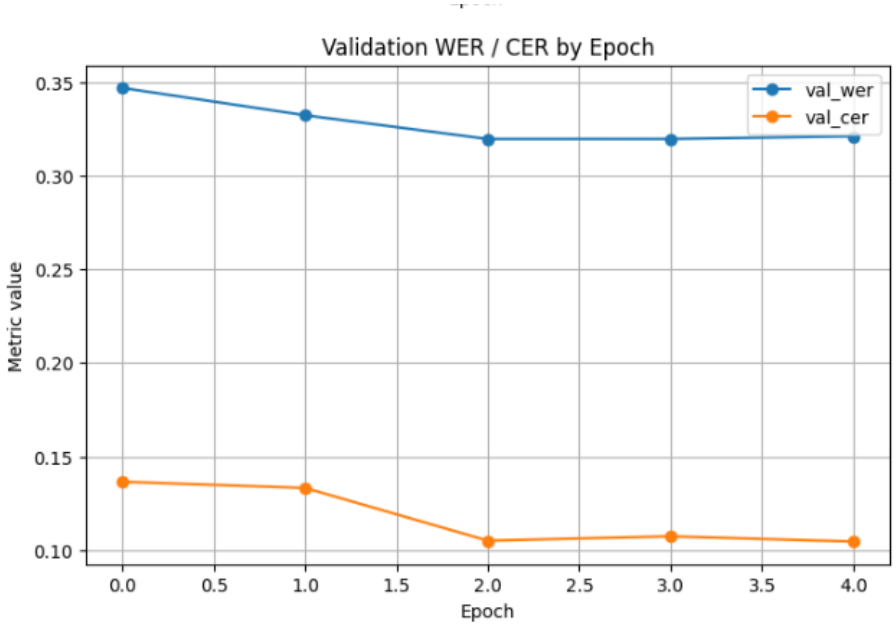

Figure 3: Validation WER and CER over 5 epochs.

The best checkpoint was saved at **epoch 2** with `val_wer = 0.3196`.

6.2 Test Set Results

We evaluated the best checkpoint on the held-out test set (21 lines, 5,517 utterances, about 9.13 hours of audio). We ran two evaluations: one using `trainer.test()` which

averages metrics per batch, and one using a custom function that collects all predictions first and then computes metrics over the whole corpus at once. The second approach is more correct for WER/CER because these are corpus-level metrics.

Table 5: Final test set results

Evaluation	WER	CER
`trainer.test()` (batch-level)	0.3715	0.1536
corpus-level (all predictions combined)	**0.3636**	**0.1446**

The main result is **WER = 36.4%** and **CER = 14.5%**. Figure 4 visualises these numbers.

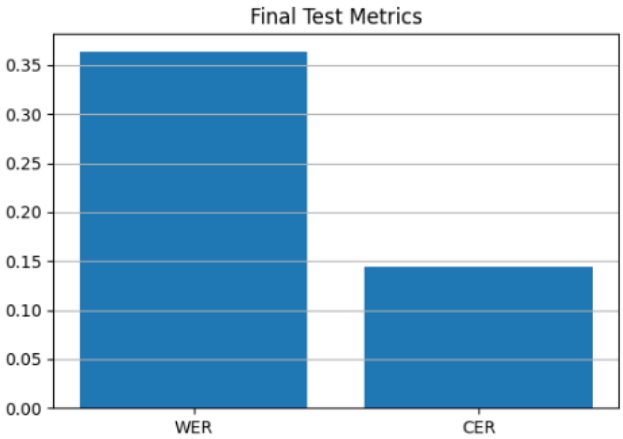

Figure 4: Final WER and CER on the test set.

6.3 Prediction Length Analysis

Figures 5 and 6 compare the character lengths of predictions and references. The distributions look almost the same (Figure 5), and the scatter plot (Figure 6) shows most points are near the diagonal, meaning the model is not systematically producing outputs that are too short or too long.

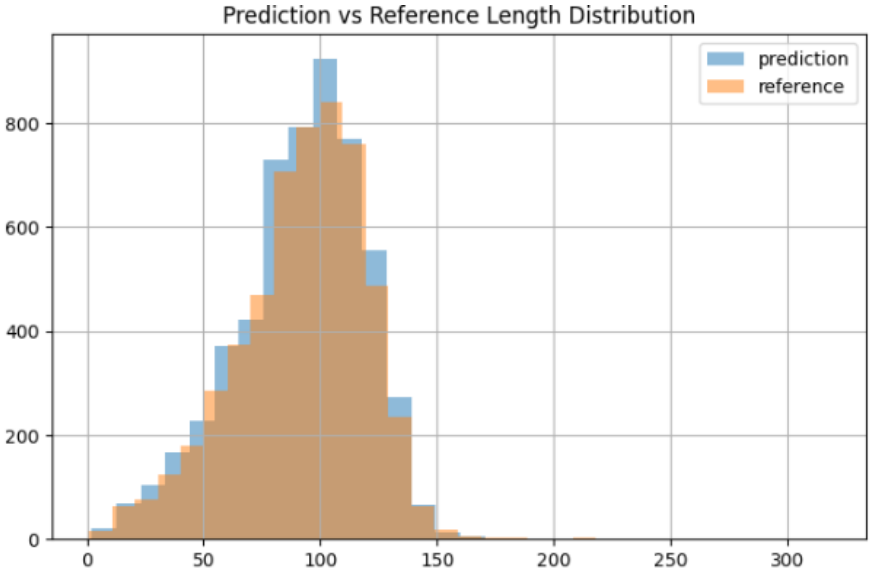

Figure 5: Prediction vs reference character length distribution on the test set.

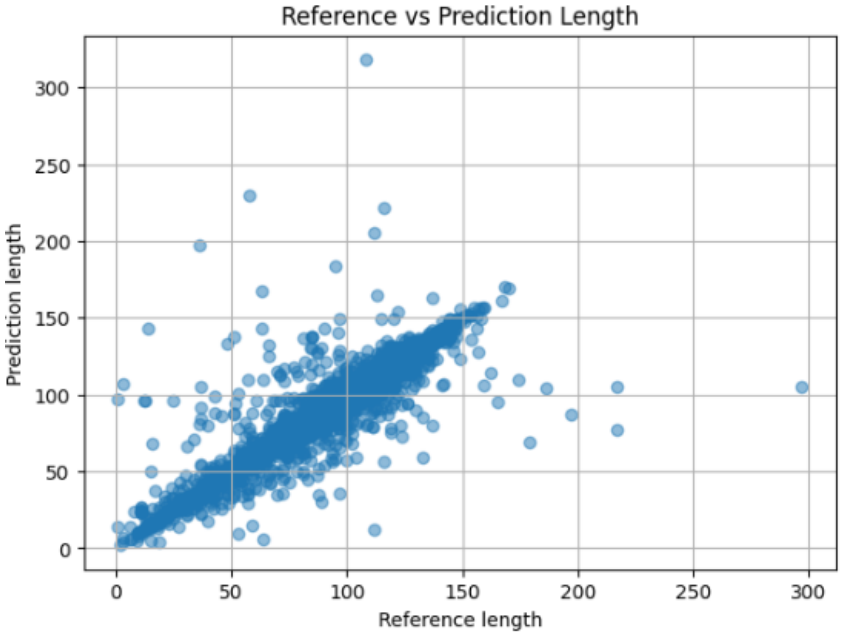

Figure 6: Scatter plot of reference length vs prediction length. Points near the diagonal mean the output length is correct.

6.4 Sample Predictions

Table 6 shows a few random examples from the test set.

Table 6: Sample reference–prediction pairs from the test set

Reference	**Prediction**
Водночас весь світ вимагає від Ердогана доказів незаконної діяльності прихильників Гюлена. Доказів немає.	Водночас весь світ вимагає від Ердогана доказів незаконної діяльності прихильників Гюлена. Доказів немає.
А до того, що навіть працівники «Київводоканалу», «Київгазу», «Київпастрансу» та «Київавтодору»	А до того, що навіть працівники Київ водоканалу, Київ газу, Київ пастрансу та Київ автодору
Тобто розробники, які грою «Відьмак-3» надихали геймдев протягом п'яти років,	Тобто, розробник, який грою від Мак-3 надихали гімнде в протягом п'яти років,
та намагався невпевнено дискутувати з колаборанткою. Класичний хард-ток від Гордона.	та намагався невпевно дискутувати з колобуранткою – класичний хардток від Гордона.

7 Discussion

WER = 36.4% means that roughly one word out of three is wrong. That sounds bad, but considering we only trained for 5 epochs on about 19 hours of data, it is a reasonable result. Out of the box, whisper-small on Ukrainian usually gives WER well above 50% on domain-specific data, so fine-tuning clearly helped.

CER = 14.5% is much lower than WER, which makes sense. A lot of "wrong" words are only slightly wrong - one letter off, different punctuation, or a different inflectional ending. These count as full word errors but only a few character errors.

Looking at the sample predictions, the main types of errors are:

- **Compound names**: "Київводоканалу" gets split into two separate words.
- **Rare words**: "Відьмак-3" becomes "від Мак-3" because the model probably never heard that word during pre-training.
- **Punctuation**: fairly random, but it barely matters for WER since punctuation is usually ignored by the metric.

The training curve (Figure 2) shows some overfitting: training loss goes down while validation loss goes up after epoch 1. Adding dropout or stronger weight decay might help. Still, WER kept improving until epoch 2 which is why early stopping did not trigger.

Limitations:

- About 11,000 audio files were missing, so we trained on less data than was available.
- No number normalization - phrases with digits likely inflate WER.
- Greedy decoding only (beam=1). Beam search would probably give lower WER.

- Only 5 epochs due to compute limits.

8 Conclusion

We fine-tuned **openai/whisper-small** on the Toronto Ukrainian dataset using PyTorch Lightning and got **WER = 36.4%** and **CER = 14.5%** on the held-out test set. The data split was done at the line level to prevent any leakage. Training used mixed-precision, gradient accumulation, and a proper checkpoint/early-stopping setup. The best checkpoint came from epoch 2. All requirements from the assignment were covered.

A Environment

Hardware: Kaggle, Tesla T4 GPU (16 GB VRAM), CUDA 12.8.